

Inteligencia Artificial

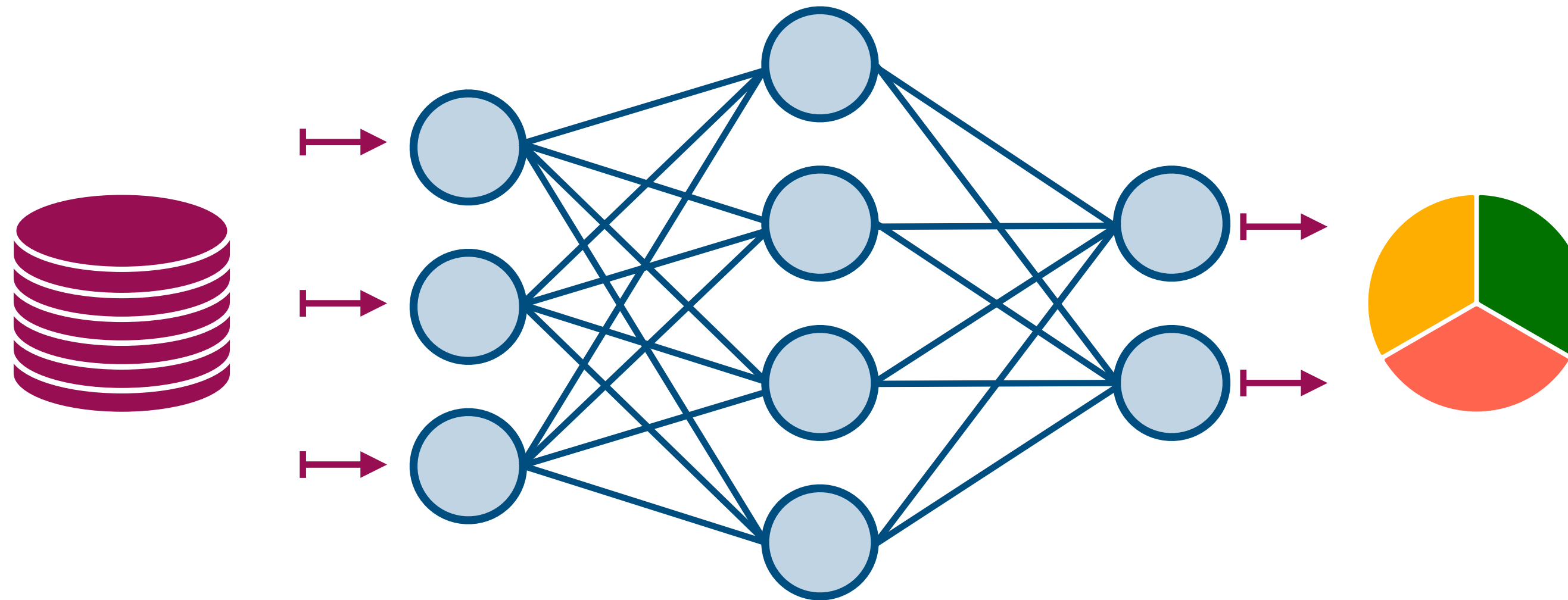
Machine Learning - (Aprendizaje Automático)

Redes Neuronales Convolucionales

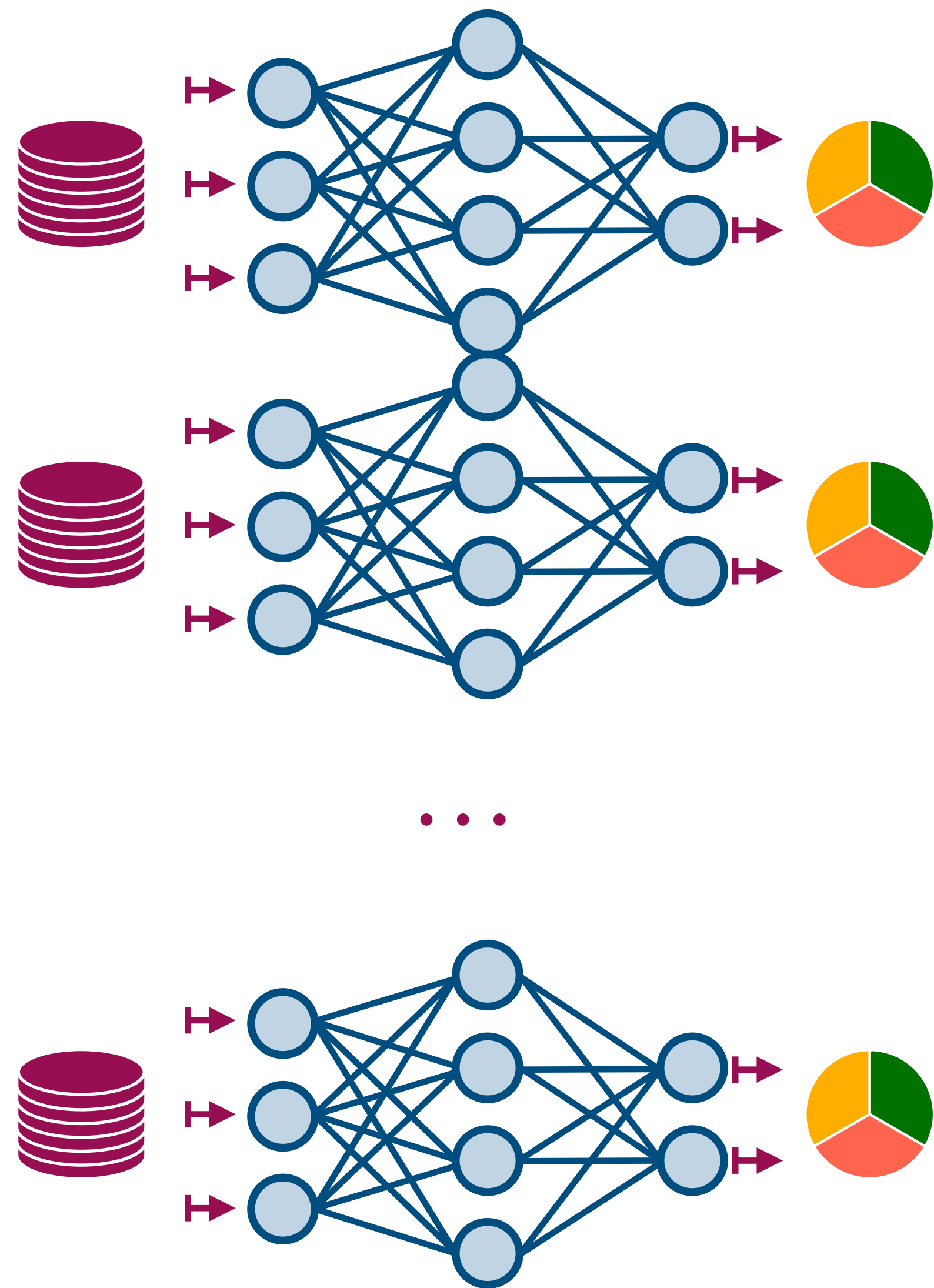
(basadas en presentaciones del MIT & Stanford Universities)



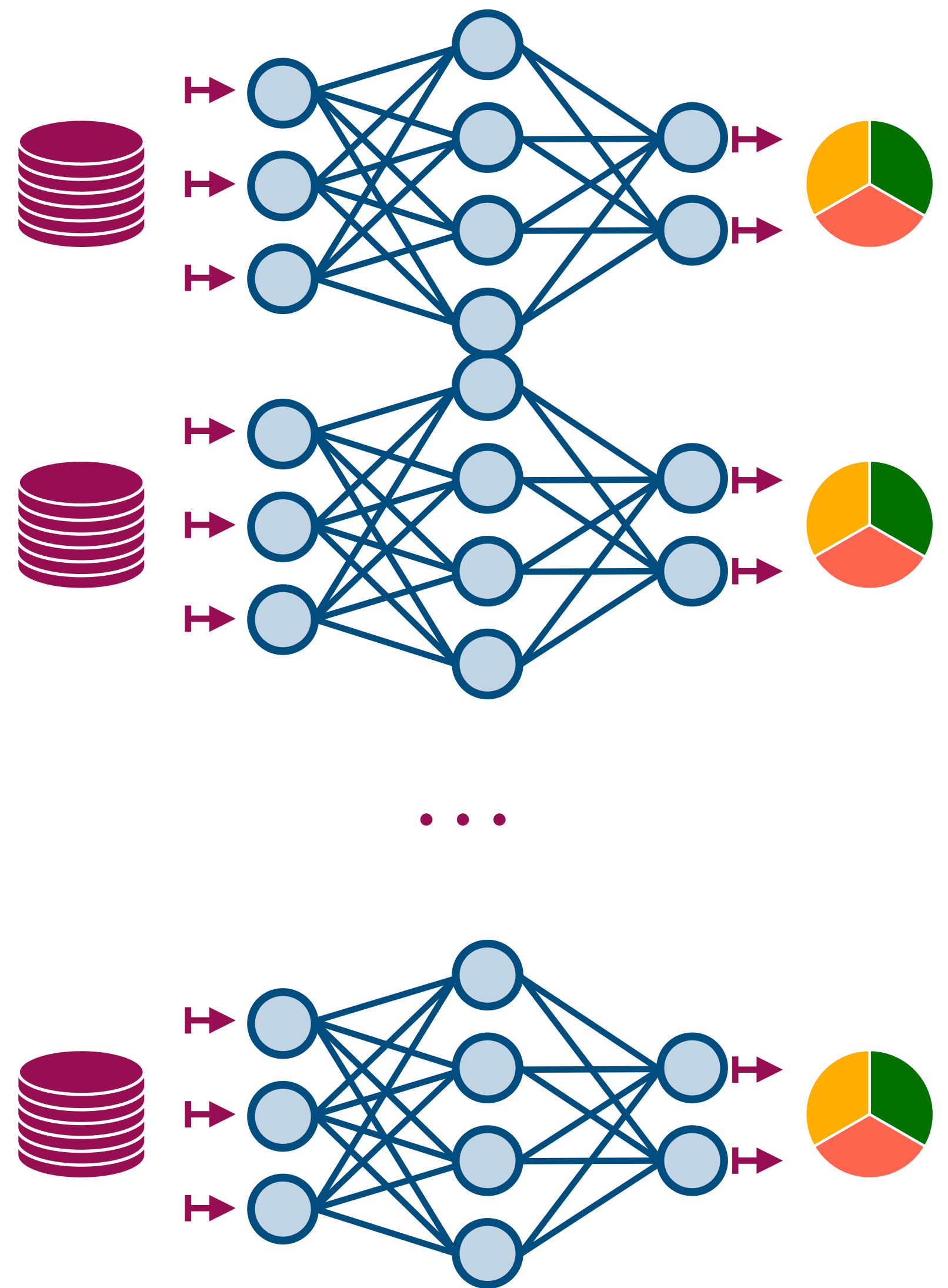
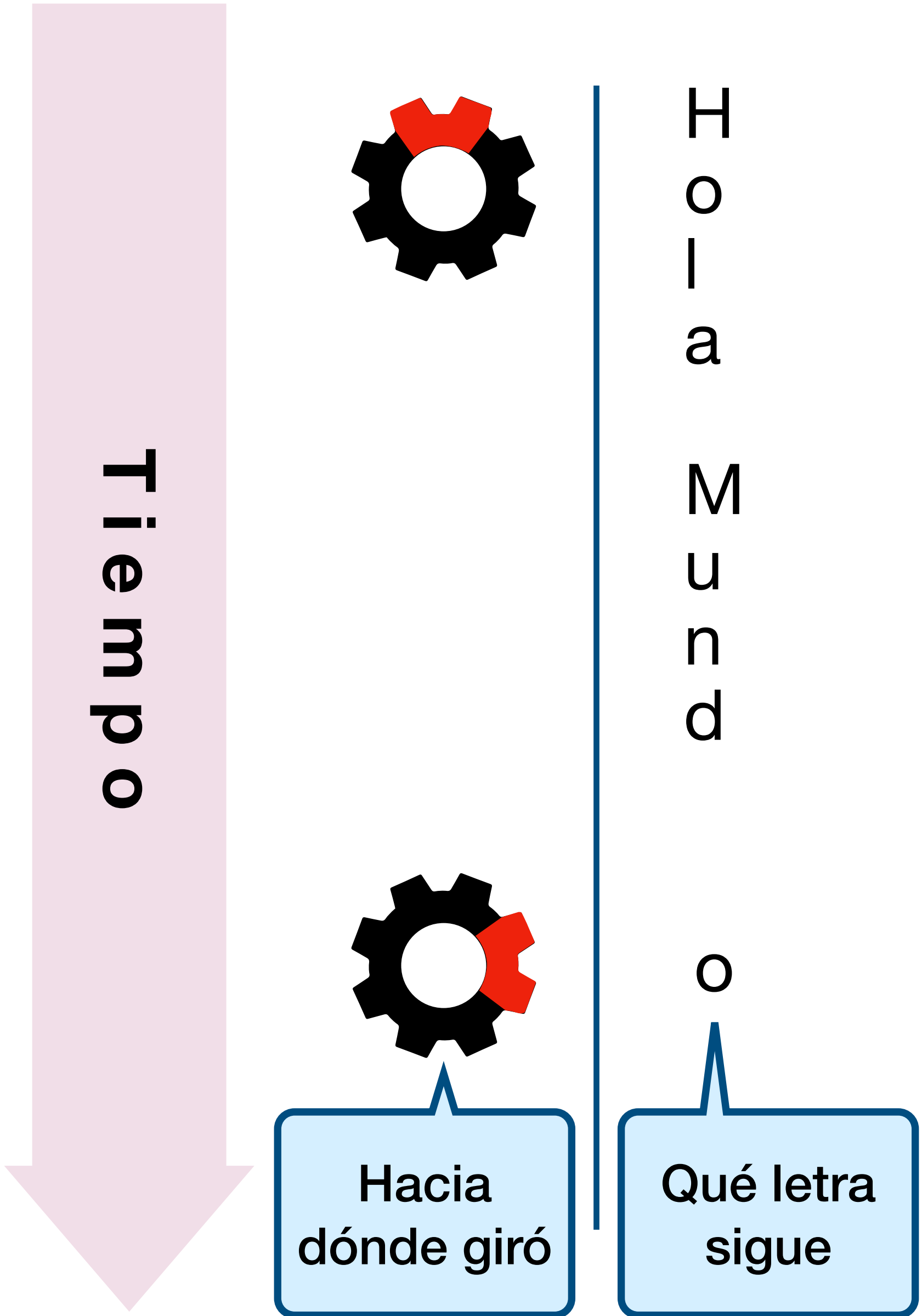
Redes Neuronales - Cómo recordar en el proceso



Redes Neuronales - Cómo recordar en el proceso

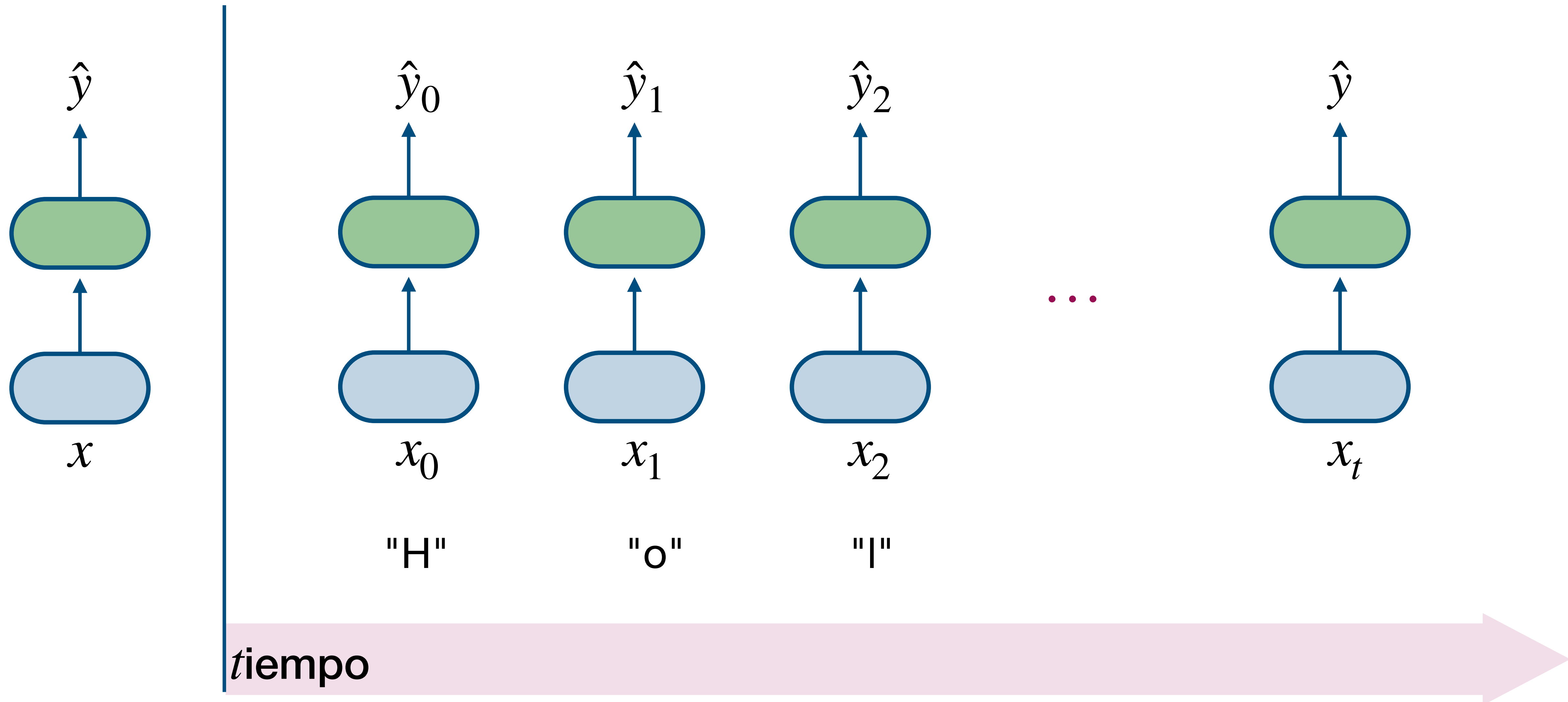


Redes Neuronales - Cómo recordar en el proceso

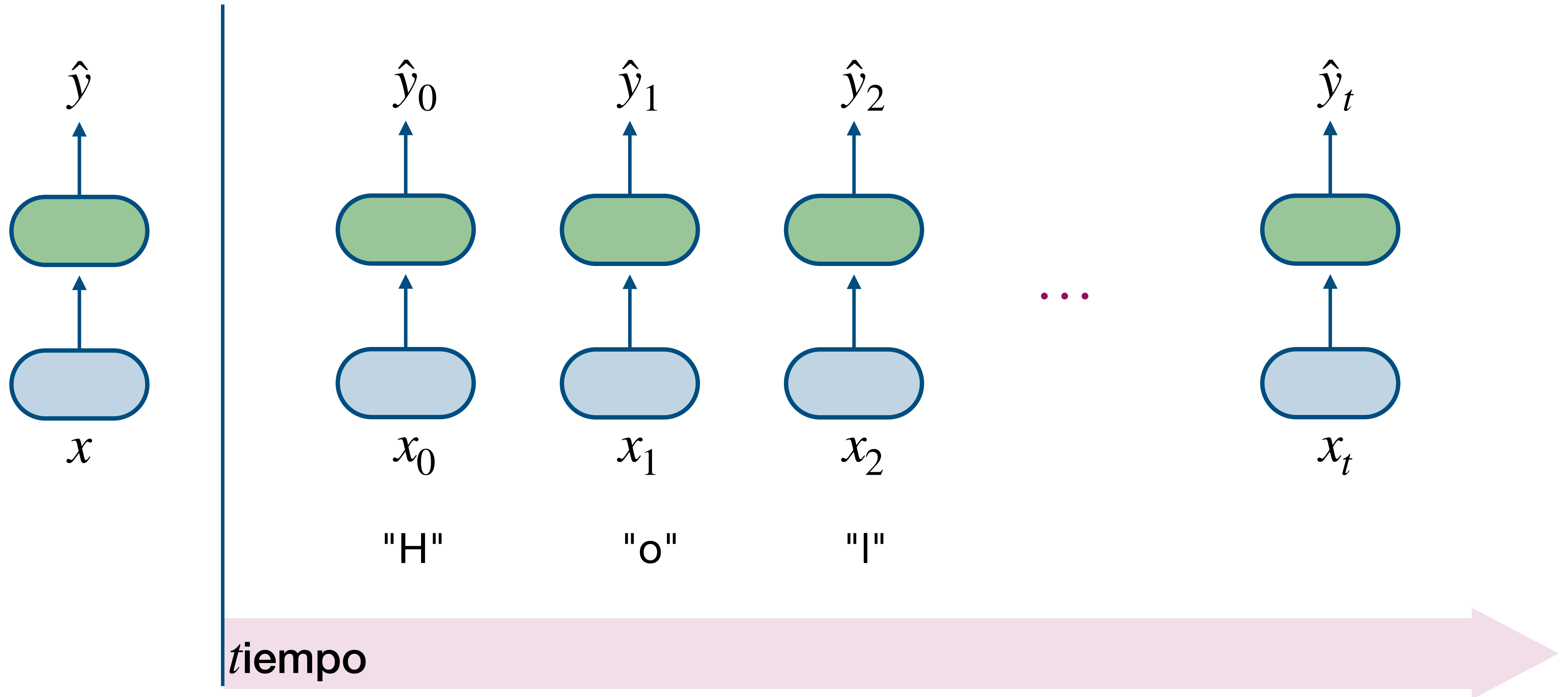


Redes Neuronales - Mirada en el tiempo

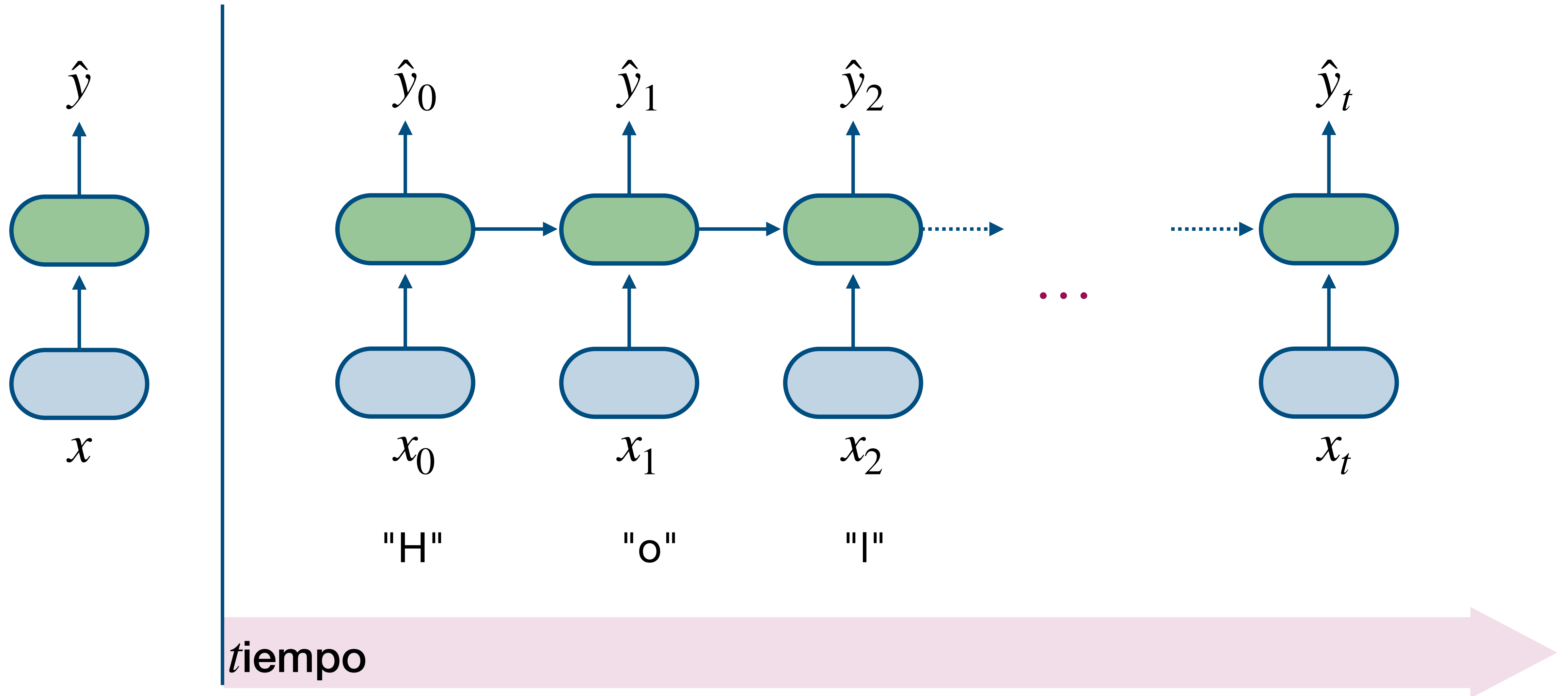
Redes Neuronales - Mirada en el tiempo



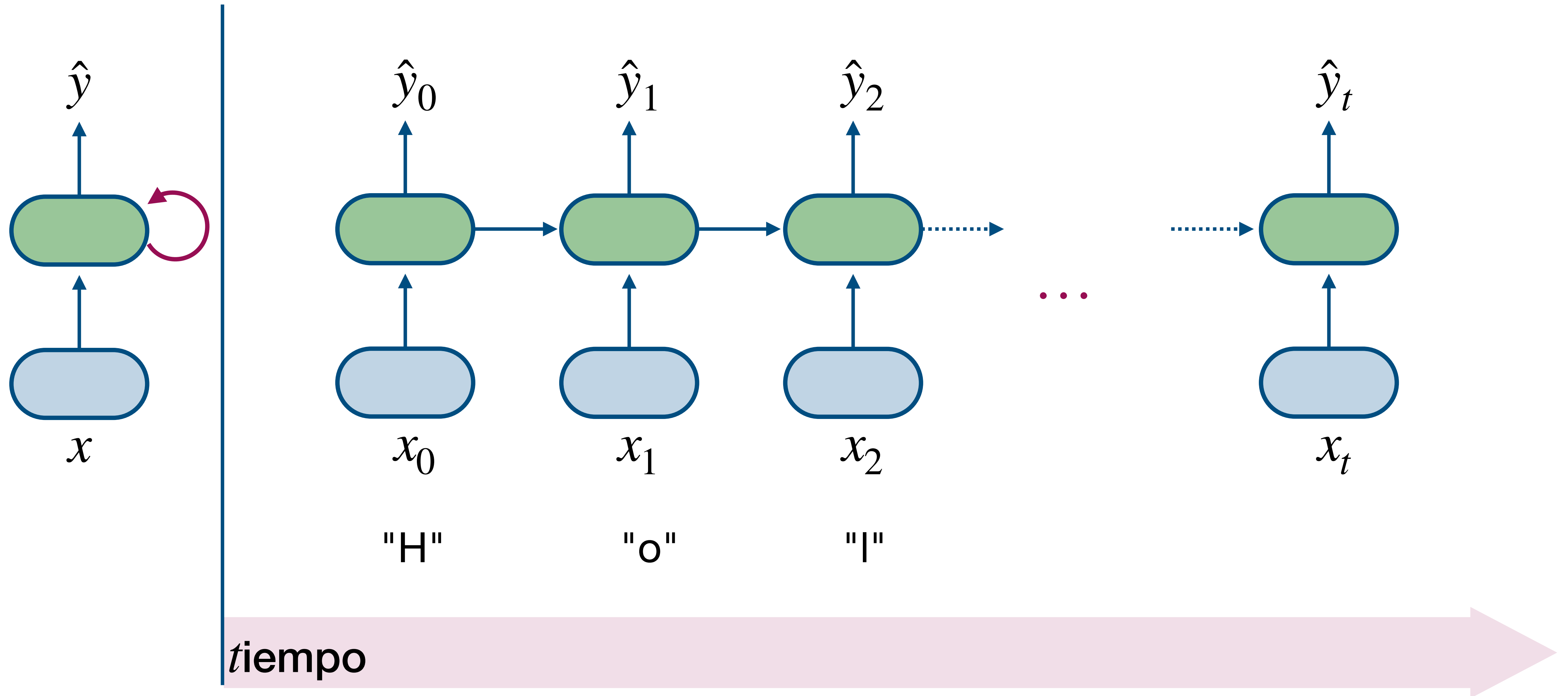
Redes Neuronales - Implementando memoria



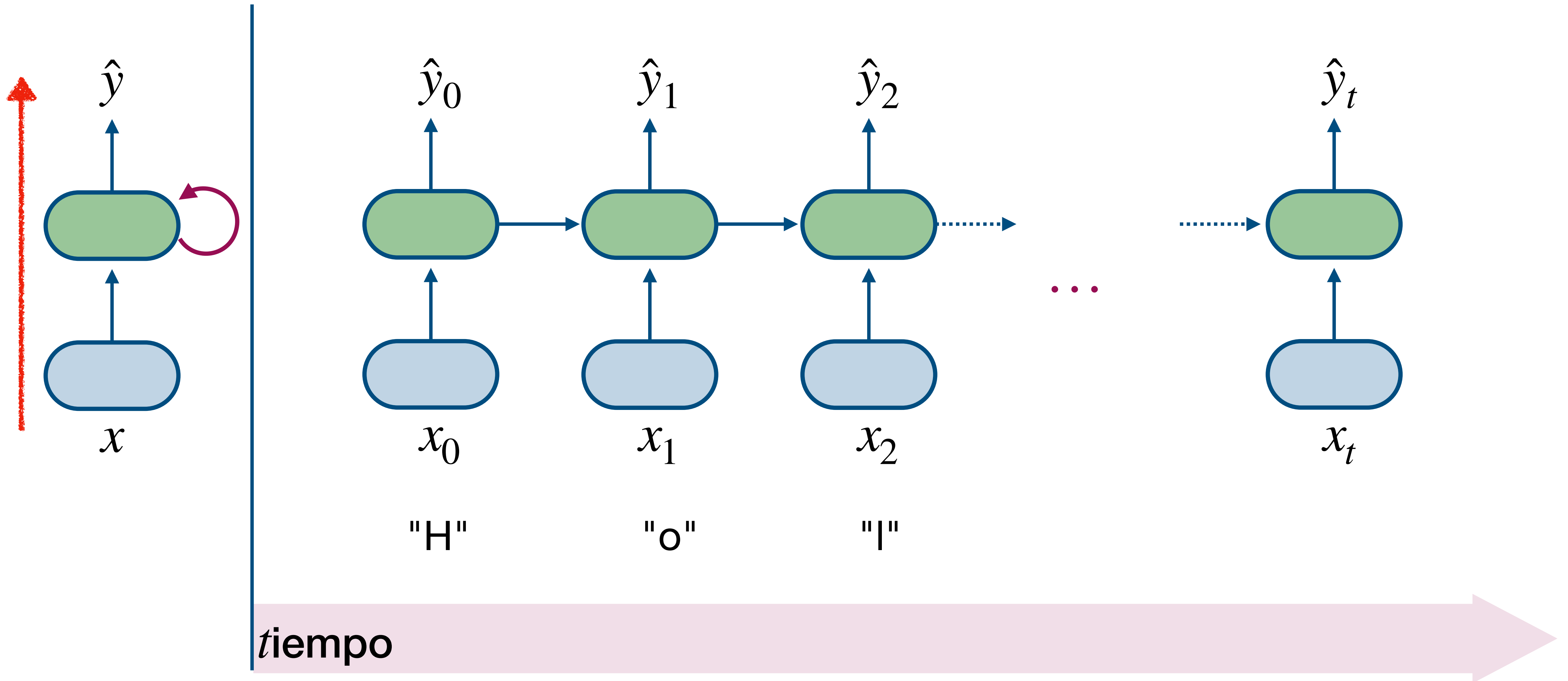
Redes Neuronales - Implementando memoria



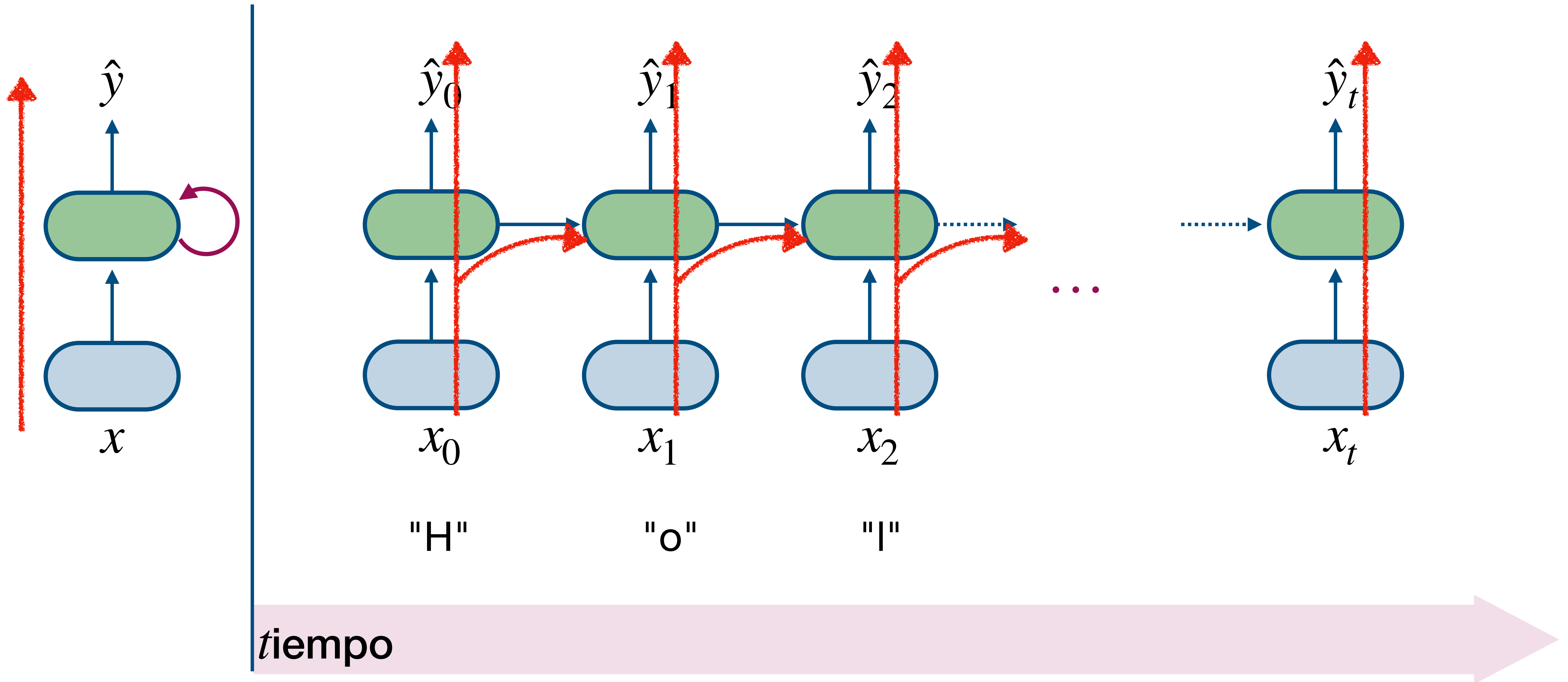
Redes Neuronales - Implementando memoria



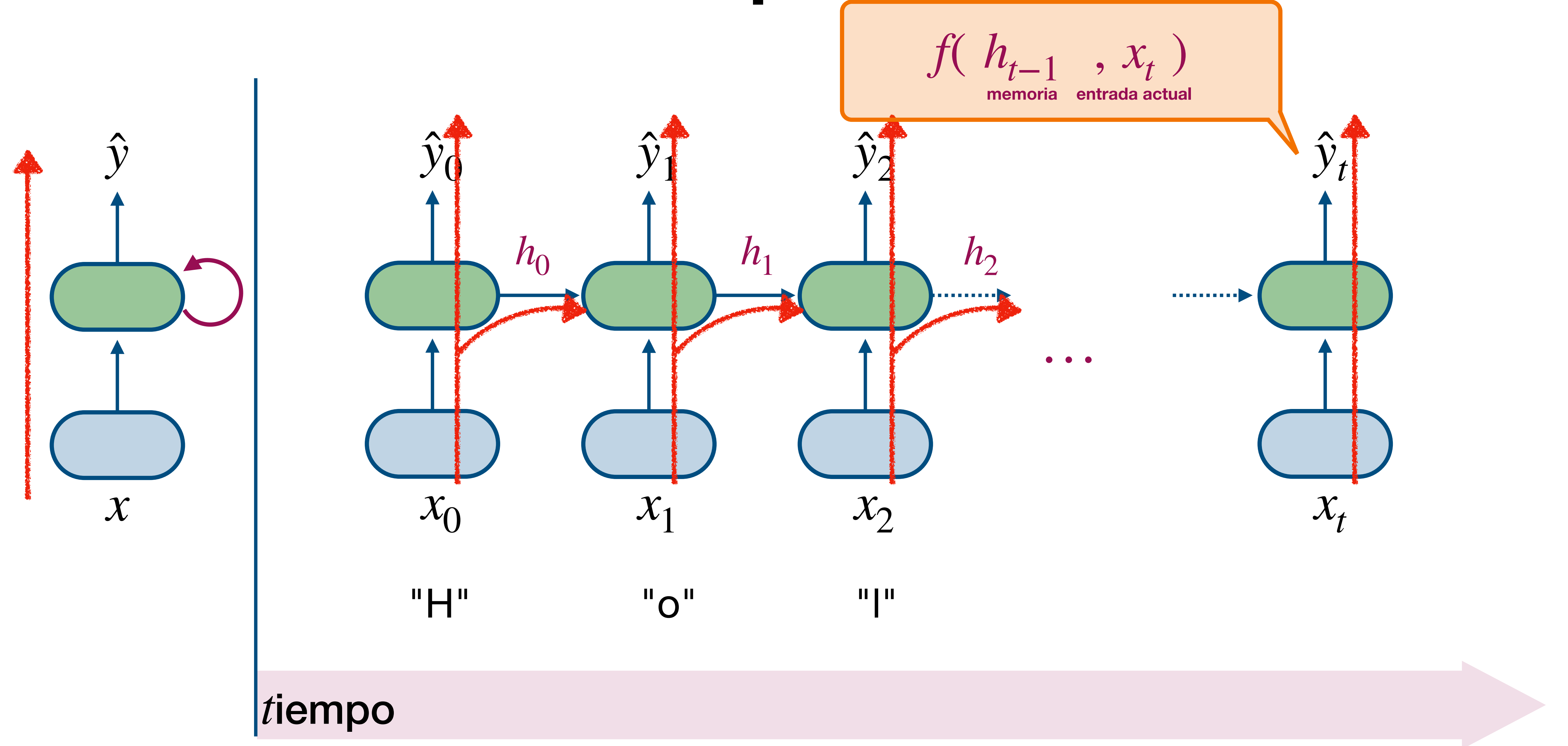
Redes Neuronales - Implementando memoria



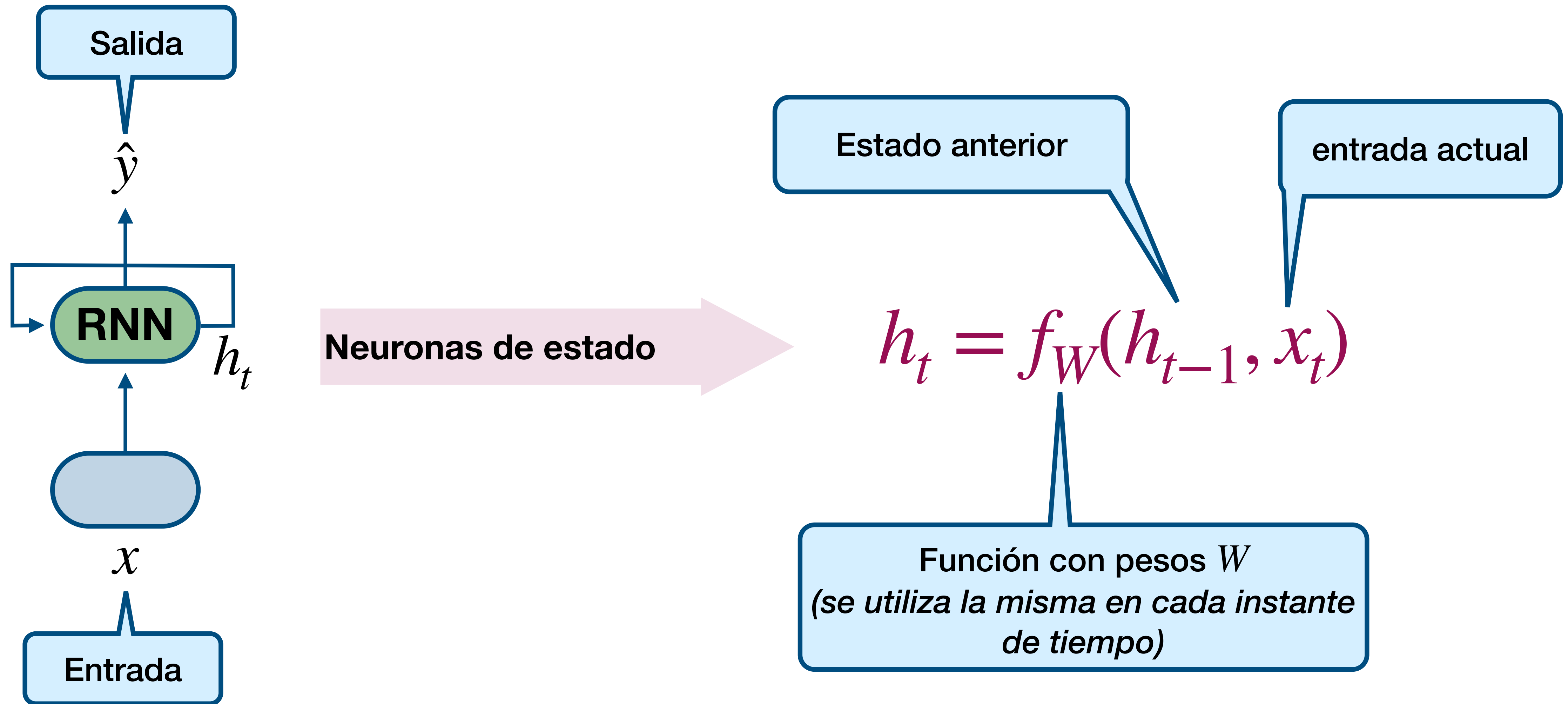
Redes Neuronales - Implementando memoria



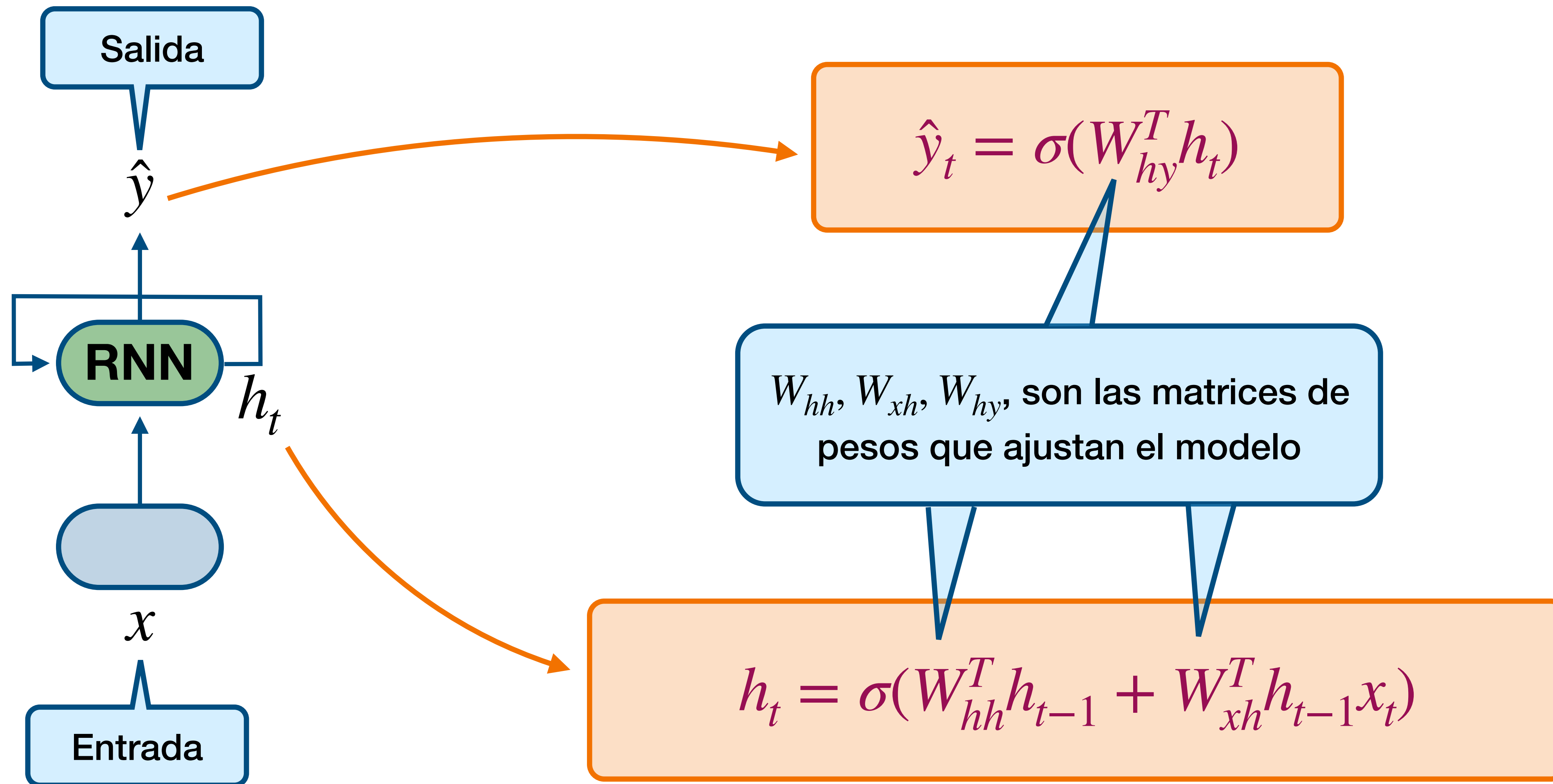
Redes Neuronales - Implementando memoria



Redes Neuronales Recurrentes (RNN)



Redes Neuronales Recurrentes (RNN)



Redes Neuronales Recurrentes (RNN)

- Una **Red Neuronal Recurrente (RNN)** es un tipo de red neuronal diseñada para procesar **secuencias de datos** donde el orden importa de los elementos. A diferencia las ANN, una RNN tiene **conexiones recurrentes** que permiten que la información fluya **de un paso de tiempo al siguiente**, implementando una forma de “memoria” temporal.
- En una RNN, el estado oculto en el tiempo t , denotado h_t , depende tanto de la entrada actual x_t como del estado oculto anterior h_{t-1} :

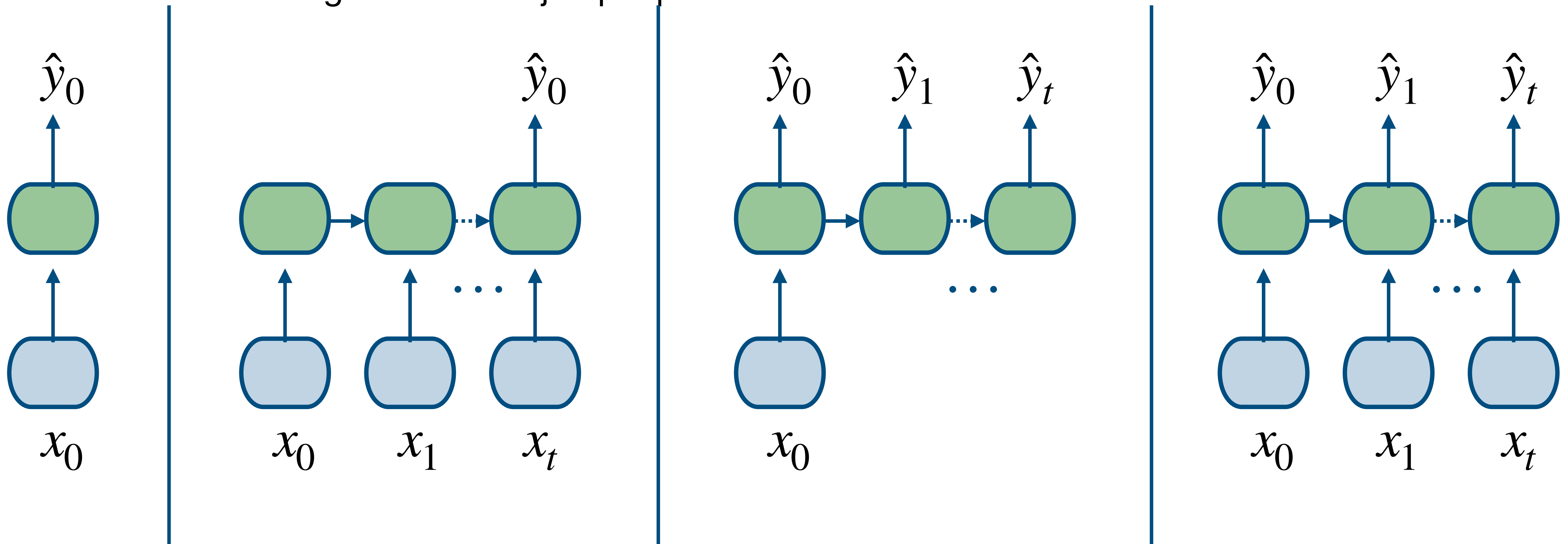
$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$
$$y_t = g(W_{hy}h_t + b_y)$$

donde:

- x_t : entrada en el tiempo t
- h_t : estado oculto (memoria)
- y_t : salida en el tiempo t
- W_{xh} , W_{hh} , W_{hy} : matrices de pesos
- f : función de activación (tanh, ReLU, ...)
- g : función de salida (softmax, lineal, ...)

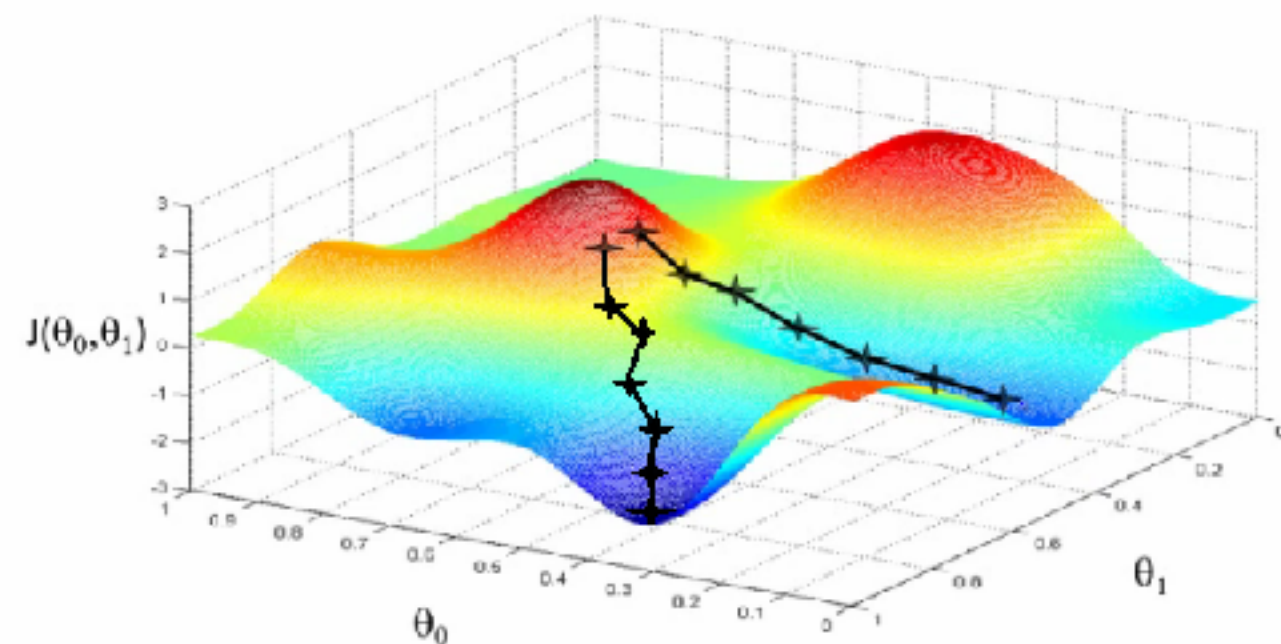
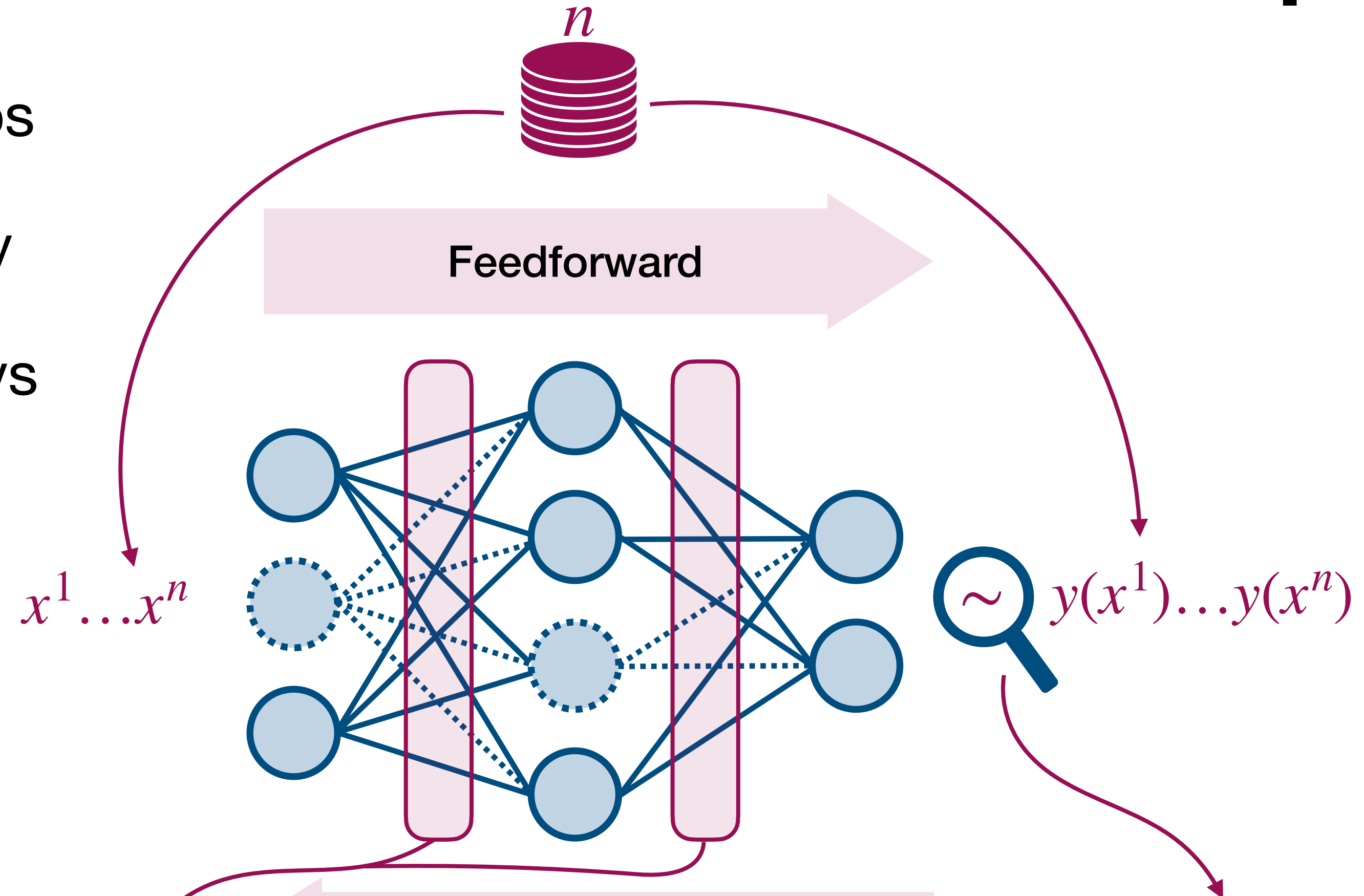
RNN - Tipos de modelos secuenciales

- **Vector a Vector:** Sin necesidad de secuencia, como una ANN
- **Secuencia a Vector:** La red “resume” la información temporal en un estado final. Por ejemplo análisis de sentimiento de un texto.
- **Vector a Secuencia:** toma una información y genera una secuencia de salida. Por ejemplo para la generación de subtítulos (captioning) de una imagen.
- **Secuencia a Secuencia:** transforma una secuencia en otra. Pueden existir variantes respecto de sincronización o longitudes. Por ejemplo para traducir de un idioma a otro.



RNN - ¿ Cómo las entrenamos ?... Recordemos para ANN

Para entrenar la red, calculamos la salida para cada caso de entrenamiento (**Feedforward**) y luego, en contraste con el resultado del caso de estudio vs el de la red, computamos los pesos y sesgos mediante la retropropagación del error (**Backpropagation**).



$$\Delta C = \frac{\delta C}{\delta w_{jk}^L} = a_k^{L-1} \sigma'(z_j^L) \frac{\delta C}{\delta a_j^L}$$

$$= 2(a_j^L - y_j) \sum_{j=1}^{m_{L+1}} w_{jk}^{L+1} \sigma'(z_j^{L+1}) \frac{\delta C}{\delta a_j^{L+1}}$$

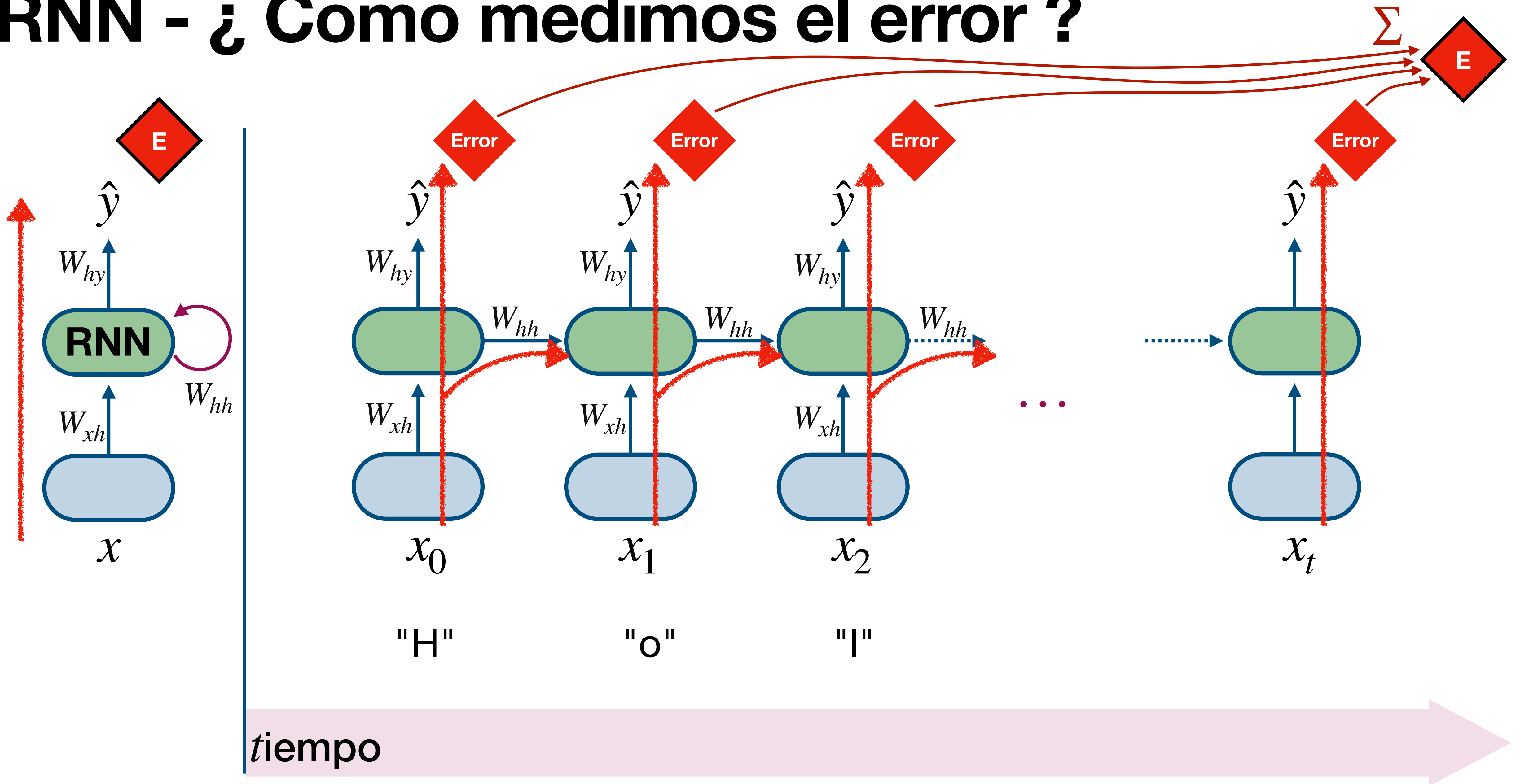
RNN - ¿ Cómo las entrenamos ? BPTT

El entrenamiento para las RNN, es similar al que utilizamos para ANN, sólo que debemos ajustar algunas cosas:

- En el proceso de **Feedforward**, debemos tener en cuenta el cómputo de h_t .
- El cómputo del error se realiza en base a una secuencia.
- La retropropagación del error (**Backpropagation**) debe tener en cuenta y ajustar la matriz de pesos de h .

A ésta técnica se la denomina Retropropagación a través del tiempo (**Backpropagation Throw Time - BPTT**).

RNN - ¿Cómo medimos el error?



RNN - Cómo se entrenan BPTT - Unfolding

Para entrenar las RNN se “desenrollan” las capas RNN de la red en el tiempo durante **T pasos**. Esto equivale a convertir la RNN en una red **feedforward** profunda con **T capas que comparten los mismos pesos**.

Para cada paso t :

- Se calcula el estado oculto h_t usando x_t y h_{t-1} .
- Se obtiene la salida y_t .
- Se evalúa la pérdida $L_t = \text{loss}(y_t, \hat{y}_t)$.

La pérdida total en la secuencia es: $L = \sum_{t=1}^T L_t$

RNN - BPTT - Backpropagation

Los errores se propagan hacia atrás a lo largo del tiempo.

- Se computa el gradiente de la pérdida L_t respecto a los parámetros.
- Como h_t depende de h_{t-1} , se debe aplicar la regla de la cadena hacia atrás:

$$\frac{\partial L}{\partial W_{hh}} = \sum_t \frac{\partial L_t}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \dots$$

- Se actualizan los pesos con el **learning rate** η y se aplica a todos los pesos compartidos (W_{xh} , W_{hh} , W_{hy}).

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

RNN - Ajustes adicionales al entrenamiento

- Se usan **minibatches** de secuencias (p. ej., 32 frases)
- Las **secuencias se truncan a una longitud fija** (por ejemplo, 50 pasos) para evitar BPTT muy largos.
- Se usa **truncated BPTT**: el error se propaga **solo por una ventana temporal fija** (por ejemplo, 20 pasos atrás).

RNN - Problemas con los gradientes

La multiplicación sucesiva de muchas derivadas puede causar:

- Si los valores son > 1 podemos tener **explosión del gradiente** (tendientes a ∞)
- Si los valores son < 1 podemos caer en **desvanecimiento del gradiente** (tendientes a 0)

Este problema deriva en la dificultad de mantener Dependencias de largo término.

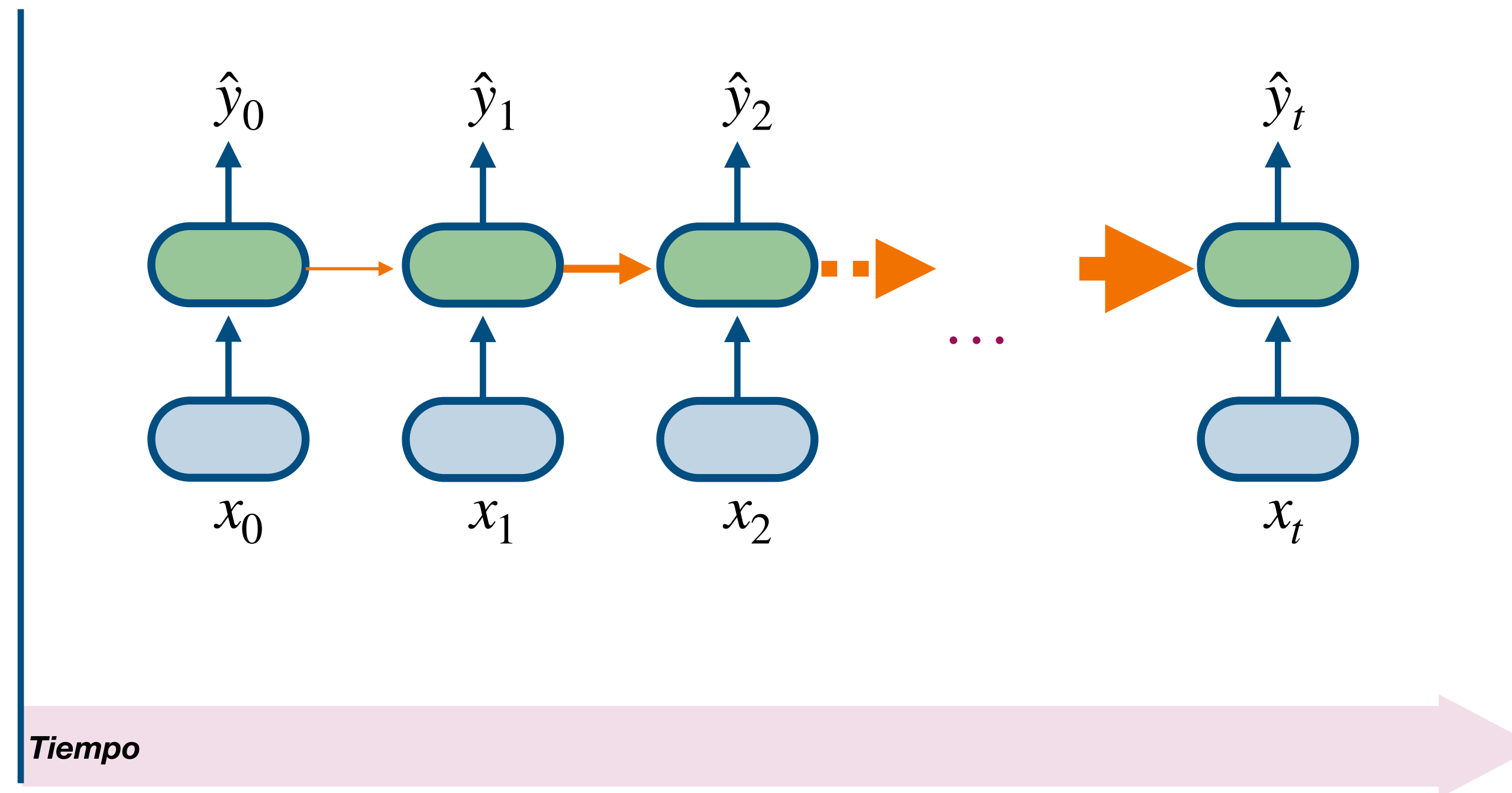


RNN - Problemas con los gradientes

La multiplicación sucesiva de muchas derivadas puede causar:

- Si los valores son > 1 podemos tener **explosión del gradiente** (tendientes a ∞)
- Si los valores son < 1 podemos caer en **desvanecimiento del gradiente** (tendientes a 0)

Este problema deriva en la dificultad de mantener Dependencias de largo término.



Long short-term memory RNNs - LSTM

LSTM (*Long Short-Term Memory*) es una arquitectura de red neuronal recurrente (RNN) diseñada para recordar información durante largos períodos de tiempo y evitar el problema del desvanecimiento del gradiente (vanishing gradient). [S. Hochreiter y J. Schmidhuber en 1997]

Implementan células (neuronas) con memoria controlada. Una célula LSTM tiene **tres puertas (gates)** y una **celda de memoria** que controlan el flujo de información:

- **Forget Gate (puerta de olvido):** Decide qué parte de la información anterior debe descartarse:

$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$ Si $f_t = 0$, se olvida completamente; si $f_t = 1$, se conserva.

- **Input Gate (puerta de entrada):** Decide qué nueva información se agregará a la memoria

$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$ Luego se crea una nueva candidata a memoria:

$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$, finalmente se actualiza la celda: $C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$

- **Output Gate (puerta de salida):** Controla qué parte del estado de la celda se usará para generar

la salida actual: $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$, luego la salida final será: $h_t = o_t * \tanh(C_t)$

Gated Recurrent Unit - RNNs - GRU

Las GRU simplifican las LSTM eliminando la celda de memoria separada fusionando algunas puertas. Esto reduce los parámetros y acelera el entrenamiento sin perder mucha capacidad de modelado [Kyunghyun Cho et al. 2014].

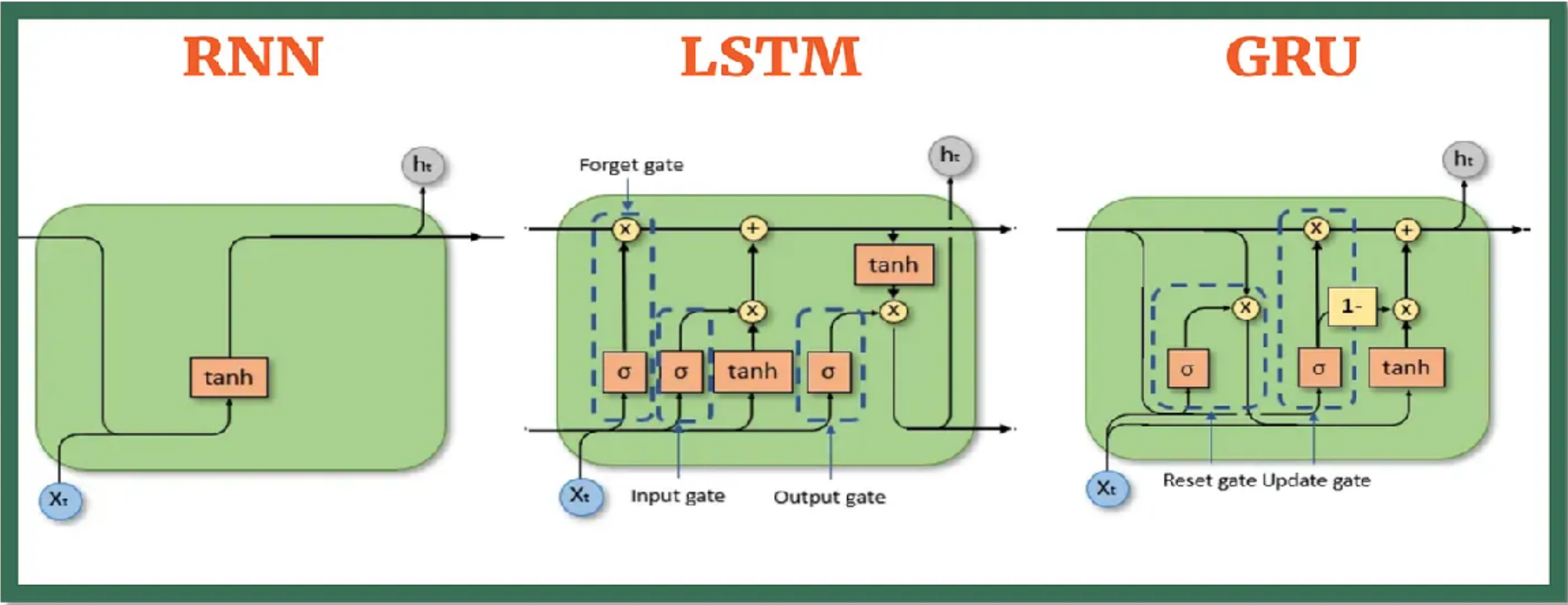
Implementan células (neuronas) que deciden cuánto recordar y cuánto olvidar con dos compuertas:

- **Update Gate (puerta de actualización):** Controla cuánto del estado anterior h_{t-1} se mantiene y cuánto se reemplaza con nueva información: $z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$.
- **Reset Gate (puerta de reseteo):** Decide cuánto del estado anterior debe olvidarse:

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

Con esta información se calcula un nuevo estado $\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$ y finalmente se actualiza el estado oculto: $h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$

Arquitectura de celdas



Característica	RNN	LSTM	GRU
Control de memoria	No	Sí (celda de memoria)	Sí (simplificada)
Puertas	No	3 (input, forget, output)	2 (update, reset)
Dependencias largas	Mala	Excelente	Muy buena
Velocidad de entrenamiento	Rápida	Lenta	Media
Complejidad computacional	Baja	Alta	Media
Requiere más datos	No	Sí	Menos que LSTM

RNN - Ejemplo prediciendo el próximo caracter (autocompletar)

"Cómo predecir el próximo o"

